NavSphere complete implementation report

11 November 2026 · ESP32S-WROOM · Six onboard ADC1 channels

1. Delivered scope

NavSphere implements a local telemetry pipeline from ESP32 firmware through authenticated MQTT, FastAPI, the existing MySQL-compatible server and a React dashboard. The user explicitly reduced the initial ten-channel request to six onboard channels with no extra acquisition hardware. The master development prompt, firmware, payload schema, simulator, database and UI reflect that change.

The software is implemented and host-tested. Hardware measurement accuracy, physical output voltages, RF behavior and long-duration stability remain pending on an actual connected board. No board was flashed. Browser visual verification could not be completed because Computer Use stopped when it could not establish the current Edge URL confidently. Docker execution is unverified.

2. Requirements and existing codebase

The initial repository contained only the master prompt and a native ESP-IDF CMake skeleton. `firmware/main/main.c` had an empty `app_main()`. No Wi-Fi implementation, event handlers, SDK target, pin assignment or credentials were provided. The original ESP-IDF/CMake foundation was extended, with no Arduino migration and no working Wi-Fi module replaced. Preservation of pre-existing Wi-Fi behavior cannot be claimed because the supplied code contained none. The detailed pre-implementation assessment is in docs/architecture/.

The user identified ESP32S-WROOM. Classic ESP32 ADC2 shares resources with Wi-Fi; the selected six exposed ADC1 GPIOs avoid that conflict. GPIO37/38 are not assumed to be exposed on the WROOM board. All six telemetry fields are logical names with an explicitly documented physical mapping.

3. System architecture

```
ESP32 ADC1 acquisition -> bounded sample queue -> MQTT publish at nominal 100 ms
                                               -> Mosquitto authenticated topics
                                               -> bounded backend ingress queue
                                               -> validation -> MySQL commit
                                               -> WebSocket received/persisted events
                                               -> React overview and analytics
Admin -> authenticated REST -> audited command -> MQTT -> ESP32 GPIO latch
                                                       <- accepted/applied/rejected ack
```

Services and artifacts are segregated into firmware/, backend/, frontend/, database/, mqtt/, simulator/, scripts/, docs/ and output/pdf/. Secrets and downloaded tools are excluded from source. The optional container topology has no MySQL service, so it cannot silently start a second database instance.

4. Firmware and FreeRTOS design

Native C11 modules separate application configuration, Wi-Fi events, ADC access, MQTT transport, command validation and output handling. The acquisition task has priority 5 and 4096 bytes of stack. The publishing task has priority 4 and 6144 bytes. Wi-Fi and MQTT reconnect workers use priority 3 and 3072 bytes each. Both periodic tasks use `vTaskDelayUntil()`. Acquisition is independent of broker availability. The statically allocated four-sample queue discards older samples under pressure and counts losses; publishing drains to the newest sample. The MQTT outbox is capped at 16 KiB with a bounded residence policy.

There is no disconnected flash telemetry journal. Offline samples are dropped and counted rather than accumulating indefinitely. Stale publication preserves the original acquisition sequence and timestamp. It cannot masquerade as a new measurement. Telemetry construction uses bounded buffers; cJSON command parsing and SDK MQTT allocations are documented exceptions to static-memory preference. No formal MISRA or CERT compliance certification is claimed.

5. ADC and output mapping

Logical signal	Physical function
analog_1	GPIO32 / ADC1
analog_2	GPIO33 / ADC1
analog_3	GPIO34 / ADC1
analog_4	GPIO35 / ADC1
analog_5	GPIO36 / ADC1 / VP
analog_6	GPIO39 / ADC1 / VN
A	GPIO25, active-high LED/output
B	GPIO26, active-high output

The ADC uses12 bit resolution,12 dB attenuation and configurable averaging (default 4 reads per channel). The driver requires usable eFuse calibration; if unavailable, values are explicitly invalid instead of claiming calibrated millivolts from an assumed reference. Six validity bits give a full mask of 63. An invalid numeric zero is not a zero-volt measurement. Floating pins can return readable but meaningless values. Attenuation is not5V protection.

GPIO25/26 initialize LOW and return LOW after observed network disconnection, clock invalidity or the configurable 60-second inactivity watchdog. Reset-time electrical guarantees require suitable external circuit design. An applied ack means the software GPIO latch write succeeded, not that an independent sensor measured the voltage or load action. No built-in LED pin was assumed.

6. MQTT communication

Topics are `navsphere/{device_id}/telemetry`, `/command`, `/ack` and `/status`. Telemetry uses QoS1, one versioned JSON message containing six millivolt values, six raw values, valid_mask, device/boot identity, unsigned32 sequence, original sample time, clock validity, uptime, stale flag, source and loss counters. SNTP supplies valid epoch time; otherwise epoch is null. Commands without a valid device clock are rejected. Per-device usernames and ACLs constrain topic access. Status and LWT are retained; telemetry, commands and acks are not.

Commands contain a unique ID, A/B key, integer0/1, issue time and expiry. Strict validation rejects extra/duplicate fields, malformed JSON, oversized or retained messages, invalid IDs and expired delivery. A bounded replay guard prevents duplicate execution. Publication stays pending until a boot-bound ack arrives. Backend command records have pending/accepted/applied/rejected/timed_out states.

Mosquitto 2.1.2 was downloaded from the publisher and extracted locally, without running the installer or registering a Windows service. Anonymous access is disabled. The tested listener is127.0.0.1:1883. TLS configuration is supported for protected deployments; plaintext loopback is the executed development mode.

7. Python backend

FastAPI modules implement authentication, devices, health, telemetry queries, commands, MQTT and database services. Pydantic validates exact six-channel schemas and numeric/time invariants. The MQTT network callback only transfers bounded messages; database work runs outside it. The default 2048message RAM queue bounds memory. A failed head insertion retries for up to 10 seconds, then drops/counts it. Continued database outage can overflow the queue. Recovery is tested; no disk spool is claimed. One Uvicorn worker owns MQTT and WebSocket state. Multiworker use requires a shared bus and rate limiter.

Clients receive explicit received and persisted events. Persisted events are emitted only after successful transaction commit. The persisted_at field is assigned just before insertion work and is not an exact commit-completion timestamp. Counters and recent frequency cover the current backend process, not lifetime database totals. Health/readiness distinguish process availability from broker/database readiness. Structured logs avoid credentials.

8. Database architecture

The existing XAMPP MariaDB 10.4.32 at 127.0.0.1:3306 is reused. Setup creates navsphere and an isolated application account. Alembic0001 records the initial schema;0002 explicitly reduces analog storage to six columns. The current telemetry table stores unsigned serial ID, device/boot/sequence, nullable epoch, server timestamps, six unsigned millivolt fields, validity mask and JSON metadata. Users, sessions, devices, commands and audit events are separate tables.

The unique device/boot/sequence index handles QoS1 repeats and reboot sequence reuse. Device/time, receive-time and retention indexes support latest/history queries. History has a maximum 31day window and 2-2000points; SQL time buckets bound returned points and preserve full-range valid min/max/average statistics. The dashboard exports plotted points, including averages when downsampled.

At 10 Hz one device produces 864,000 rows/day. Measure actual table/index growth before selecting retention. A dry-run maintenance command reports candidates; explicit execution deletes eligible telemetry in bounded batches. Backups use single-transaction mysqldump of NavSphere only; a real local backup was generated successfully. Automatic retention scheduling was not enabled.

9. Frontend architecture and behavior

React 19, TypeScript, Vite, Recharts and local Inter font assets provide a responsive industrial dashboard. Hooks manage authenticated sessions, CSRF, bounded600sample browser histories and reconnecting WebSockets. Pages cover login, overview, six dedicated analytics routes, Account and admin Controls. Cards show physical mappings, validity, original sample freshness, sparklines and receipt/persistence distinctions. Dark mode is default, with light mode.

Analytics supports live/history, local timestamps, UTC CSV, common time ranges, custom ranges, brush zoom/pan and full-range statistics. Invalid values are chart gaps. Loading, unavailable, stale, interrupted and error states are explicit. Simulator source is labelled. Commands are never optimistically displayed as applied; status and returned controls reflect backend evidence.

10. Security architecture

Passwords use salted scrypt. Random session tokens are stored as hashes and sent through HttpOnly/SameSite cookies. Mutations require per-session CSRF. Role checks protect commands, and the initial development password must change before actuation. Login attempts are rate-limited, sessions expire/revoke, and administrative activity is audited. OpenAPI declares cookie auth and CSRF input. Production settings reject default admin credentials, insecure cookies, plaintext MQTT, SQLite and non-HTTPS frontend origins. Startup also refuses active stored accounts that still verify against the default development password, including accounts retained when promoting a development database.

Local .env/NVS provisioning keeps secrets out of source. Development NVS is not encrypted; production needs reviewed secure boot/flash/NVS encryption and trusted TLS certificates. No irreversible eFuse changes were performed. MQTT messages rely on authenticated scoped broker identities rather than independent application signatures. Deployment hardening and real electrical validation remain operator responsibilities, not completed certification claims.

11. Installation and configuration

Use scripts/setup_windows.ps1, then scripts/start_local.ps1. They download portable tools, install Python/frontend dependencies, preserve existing local configuration, generate random service credentials and apply migrations. Detailed commands are in the separate deployment/user guide. All development services bind to loopback. LAN exposure/firewall changes are explicit operator steps. The prompt's192.168.9.59 is configurable; the adapter observed on this laptop was192.168.31.50. The dashboard and docs are at localhost5173 and 8000.

The master prompt and component documents specify the final six-channel scope. Firmware settings are centralized in app_config.h and Kconfig; secrets are provisioned into the navsphere NVS namespace. Backend uses environment variables, the frontend uses Vite variables/proxy settings, and broker paths are generated for the local workspace. No secret value is included in this report.

12. Test and performance evidence

The automated backend suite passed 28 tests including validation, auth/CSRF, roles, persistence, deduplication, WebSocket delivery, command state transitions, status/safe-state updates and database failure recovery. Ruff and Python compilation passed. These unit tests use isolated SQLite databases; they do not substitute for the separately executed MariaDB test.

The frontend suite passed 15 tests across API, pages and live WebSocket behavior. The production TypeScript/Vite build passed. The dependency audit reported zero known vulnerabilities at the executed check. Visual browser QA

remains incomplete. Firmware compiles against installed ESP-IDF 6.0.2 for classic ESP32. Host C tests cover 13 command-rule assertions and actual parser/dedupe/queue/watchdog logic. The simulator suite passes 10 tests; process-tree shutdown was tested with a sacrificial parent/child and with the live stack, leaving Apache/MySQL running. Firmware hardware accuracy, measured scheduling jitter and 24hour soak are not yet tested.

The final live integration run stored 300 of 300 synthetic records over 30.002 s (9.999 Hz), with zero simulator drops. It verified six channels in MariaDB, received/persisted WebSocket messages, all six histories, A/B on/off acks, malformed-message counting, QoS1 deduplication, authorization, CSRF, password change, logout, Swagger/ReDoc/OpenAPI and readiness. The current run's exact measurements are recorded in docs/testing/integration-results.json.

In that run, sample-to-insert-timestamp latency was median 1 ms, p95 3 ms, maximum 19 ms. Observed latest committed reading age via REST was median 56.63 ms, p95 104.11 ms. These are same-host synthetic measurements, include scheduling effects, and do not measure sensor/RF latency. Target 10 Hz and p95 committed-reading age under 250 ms at one simulated device were met in this run. No multi-device capacity or hardware CPU/heap endurance claim is made.

13. Phase delivery record

Phase	Concrete outputs and verification
1 Assessment	Inspected all original firmware files; assessment in docs/architecture; identified empty app and later confirmed board
2 Architecture	Six-channel schema, topic contract, GPIO map, configuration and safe states defined
3 Firmware	Native ADC/Wi-Fi/MQTT/commands implemented; IDF build and host C tests executed; board tests pending
4 Broker	Portable Mosquitto, PBKDF2 passwords, scoped ACLs; authenticated live connection tested
5 Backend	FastAPI/MQTT/WS/auth modular source;28 tests and Ruff passed
6 Database	Alembic head0002 on existing MariaDB; actual six-column storage/deduplication and backup tested
7 Frontend	React six-card dashboard, six analytics routes, admin/account;15 tests and build passed
8 Integration	Real broker/backend/MariaDB path plus synthetic device and command acks tested
9 Measurement	30second10 Hz test; measured throughput and read-age; no hardware benchmark claimed
10 Delivery	Separate Markdown/PDF reports, guides, test evidence, native scripts and optional Compose

Commands include `idf.py build` via firmware/tools/build.ps1, firmware host tests, `python -m pytest -q`, `python -m ruff check app tests migrations`, `npm test`, `npm run build`, `python scripts/integration_test.py`, `python scripts/backup_database.py` and the native Windows start helpers. The next implementation step is physical provisioning and the hardware test plan.

The final firmware binary is 988,272 bytes, leaving about 69% of the 3 MiB application partition free. Link-time IRAM is 87,999/131,072 bytes (67.14%); static DRAM is 44,511/180,736 bytes (24.63%). Runtime heap and stack margins require hardware measurements. Build evidence and SHA256 are recorded in docs/testing/build-results.json and output/firmware/manifest.json.

14. Limitations, troubleshooting and future work

Physical ADC calibration/linearity, actual output levels, reset behavior, 10 Hz jitter, Wi-Fi recovery and heap/stack soak must be verified on the board. No connected serial device was detected. An end-to-end simulator cannot prove these. Browser rendering remains unverified by the agent because its UI tool stopped on URL policy. Docker was not installed, so Compose was not executed.

Database/broker startup failures appear in readiness and local logs. Check credentials and actual ports rather than treating phpMyAdmin as the database endpoint. Invalid/stale cards require valid sensors and timestamps, not just a connected MQTT socket. Commands can fail on first-login restrictions, CSRF, stale devices, clock synchronization, expiry or unavailable broker. The output watchdog is intentional. The deployment guide has reproducible troubleshooting and manual QA instructions.

Future improvements can include measured hardware calibration characterization, an endurance test, persistent offline spooling if required, shared event state for multiworker deployments, managed certificate rotation, user-management UI, and workload-based retention/aggregation. These are identified extensions rather than completed features.

15. Primary references

• Espressif ADC oneshot driver

• Espressif ADC calibration

• ESP32-WROOM-32 datasheet

• Mosquitto authentication

• Mosquitto configuration

• Node official downloads

Firmware API choices were also checked against installed ESP-IDF headers.